

Phase 1 — API Auth & Security Foundations (design)

Field	Value
Date	2026-05-06
Phase	1 of 6 (parent: docs/todos/Lava_TODOs_001.md)
Status	Approved (brainstorm 2026-05-06) — pending plan
Owner	Operator + Claude
Predecessor	post-Ktor cleanup chain ending at lava-api-go-2.0.16
Successor	Phase 2 (multi-provider streaming search; absorbs alice-bug routing fix)

1. Decomposition reference

The TODO at docs/todos/Lava_TODOs_001.md covers 17 work items. They cannot fit one spec. Decomposition agreed during brainstorm:

#	Phase	TODO sections covered
1	API auth & security foundations (<i>this spec</i>)	"Lava API secure access" + transport mandate (HTTP/3, brotli, HTTPS body); + a hotfix to hide unsupported providers
2	Multi-provider streaming search	"Search request" + "Receiving results" + per-provider routing (closes alice-404) + events transport
3	First-run onboarding wizard	"Client app first run"
4	Sync subsystem expansion	"Sync now button" + "Additional sync options"
5	UI/UX polish bundle	Menu screen + Color themes + About dialog + Credentials screen + result filtering + nav-bar overlap
6		

#	Phase	TODO sections covered
	Testing sweep + distribution + documentation	"Testing" + Firebase distribute + thinker.local boot + diagrams/manuals + green-light-to-operator

2. Motivation

Today the `lava-api-go` API has no authentication: any LAN host (or, given the appropriate firewall hole, any internet host) can call its endpoints. The Android client embeds no proof-of-origin in its requests; the API cannot tell a Lava client from `curl`. The TODO mandates: (a) UUID-based client allowlist enforced server-side; (b) progressive backoff on invalid credentials; (c) HTTP/3 with brotli compression and TLS-encrypted bodies for every request and response; (d) build-time UUID injection on the Android client with encrypted-at-rest representation in the APK and decrypt-on-use lifetime in memory.

Phase 1 introduces this auth foundation. Every later phase's networked surface (Phase 2's streaming search, Phase 4's sync, Phase 6's distribution gate) consumes Phase 1's transport.

3. Goals

- **G1** Every API request from the Android client carries a per-build UUID, transported via a config-named HTTP header. The API rejects requests without a valid header.
- **G2** The API stores the allowlist as HMAC-SHA256(UUID, LAVA_AUTH_HMAC_SECRET) at boot. Plaintext UUIDs are zeroized after the boot-time hashing pass.
- **G3** Invalid headers from a given source IP trigger a fixed-ladder progressive backoff (2s → 5s → 10s → 30s → 1m → 1h ...), configurable via `.env`.
- **G4** Per-release UUID rotation: each Android release gets a unique UUID. Active and retired releases are separated; retired releases receive HTTP 426 Upgrade Required (no backoff).
- **G5** Transport mandates: HTTP/3 (preferred, with HTTP/2 fallback), brotli response compression, TLS 1.3 minimum.
- **G6** Client-side encrypted UUID at L2 layering — AES-GCM keyed by HKDF over SHA256(signing-cert)[:16] || pepper. UUID handled as `ByteArray` end-to-end; never as `String`. Decrypted on the `OkHttp` interceptor thread; zeroized immediately after request emission.

- **G7 Constitutional clause §6.R — No-Hardcoding Mandate** added to root CLAUDE.md + AGENTS.md + every submodule's CLAUDE.md. Pre-push enforcement via scripts/check-constitution.sh.
- **G8 α -hotfix:** Onboarding hides providers whose TrackerDescriptor.apiSupported = false; existing installs with such a provider selected receive a one-time dialog directing to another provider.

4. Non-goals (out of scope for Phase 1)

- Multi-provider routing / per-provider search endpoints — Phase 2.
- Streaming results (SSE/WS/gRPC) — Phase 2.
- Onboarding wizard redesign — Phase 3.
- Sync subsystem expansion — Phase 4.
- Color themes / About / Credentials redesign — Phase 5.
- .env SIGHUP hot-reload — Phase 6 operations concern.
- Per-installation UUIDs (each device its own) — explicitly rejected during brainstorm; the operator's TODO describes per-build UUIDs.
- Native (NDK) client-side decryption (L3) — explicitly deferred; L2 is the chosen layer.
- Reversible-encryption-at-rest of API allowlist (β) or KDF-hardened comparison (γ) — explicitly rejected during brainstorm.

5. Configuration model (§6.R compliance)

Single .env at repo root (existing pattern; .gitignore'd per §6.H). New variables introduced by Phase 1:

```
# === Auth field identifier (read by both API + Android build) ===
LAVA_AUTH_FIELD_NAME=Lava-Auth                # the HTTP
header name                                     #

(placeholder; the .env decides)

# === Allowlist + rotation (API reads at boot; Android build picks
one entry) ===
LAVA_AUTH_CURRENT_CLIENT_NAME=android-1.2.7-1027    # which
UUID this build embeds
LAVA_AUTH_ACTIVE_CLIENTS=android-1.2.7-1027:UUID-
A,android-1.2.6-1026:UUID-B
LAVA_AUTH_RETIRED_CLIENTS=android-1.2.5-1025:UUID-C

# === API-only ===
LAVA_AUTH_HMAC_SECRET=<32-byte base64>            # boot-time
secret for hashing
```



```

    |      2. parse LAVA_AUTH_ACTIVE_CLIENTS →
[]ClientEntry |
    |      parse LAVA_AUTH_RETIRED_CLIENTS →
[]ClientEntry |
    |      3. for each entry: hash = HMAC-SHA256(uuid,
secret) |
    |      4. zeroize plaintext UUID byte slice
(runtime.KeepAlive) |
    |      5. install middlewares (in
order): |
    |      backoffMW → authMW → brotliMW →
handlers |

```



```

| internal/auth/middleware.go
(NEW) |

|
| func AuthMiddleware(active, retired
map[string]string, |
| secret
[]byte, |
| backoff *Backoff) gin.HandlerFunc
{ |
| return func(c *gin.Context)
{ |
|     hdr :=
c.GetHeader(LAVA_AUTH_FIELD_NAME) |
|     if hdr == ""
{ c.AbortWithStatus(401); ... } |
|     blob, err :=
base64.StdEncoding.DecodeString(hdr) |
|     if err != nil
{ c.AbortWithStatus(401); ... } |
|     hash := hmacSHA256(blob,
secret) |
|     if name, ok := active[hash]; ok
{ |
|         c.Set("client_name",
name) |
|         backoff.Reset(c.ClientIP())
|
|         c.Next()
|
|         return
|
|     }

```

```

    |         if name, ok := retired[hash]; ok
{
    |         |
    |         c.Header("Lava-Min-Version-
Name",
    |
    |         config.MinSupportedVersionName)
    |         |
    |         c.JSON(426,
gin.H{
    |         |
    |         |         "error":
"client_version_unsupported",
    |         |
    |         |         "min_supported_version_name":
    |         |         |
    |         |         config.MinSupportedVersionName,
    |         |         |
    |         |         "min_supported_version_code":
    |         |         |
    |         |         config.MinSupportedVersionCode,
    |         |         |
    |         |         "client_name":
name,
    |         |
    |         })
    |
    |         c.Abort()
    |
    |         return

    |     }

    |         backoff.RecordFailure(c.ClientIP())
    |         |
    |         c.AbortWithStatusJSON(401, gin.H{"error":
"unauthorized"})|

    |     }

    | }

```



```

| internal/auth/backoff.go
(NEW)

|
|     type Backoff struct
{
|     |     steps []time.Duration // from BACKOFF_STEPS
env
|     |     state sync.Map        // ip →
*State

```

```

|   }
|
|   type State struct
|   {
|       mu
|       sync.Mutex
|       failures
|       int
|       blockedUntil
|       time.Time
|   }
|
|   func (b *Backoff) CheckBlocked(ip string) (bool,
|   int)
|       // returns (blocked,
|       retryAfterSeconds)
|
|   func (b *Backoff) RecordFailure(ip
|   string)
|       // advances ladder; clamps at last
|       step
|
|   func (b *Backoff) Reset(ip
|   string)
|
|   func BackoffMiddleware(b *Backoff)
|   gin.HandlerFunc
|       // before authMW: returns 429 + Retry-After when
|       blocked

```

internal/auth/multiprovider.go (existing) is untouched by Phase 1; per-provider concerns live in Phase 2.

submodules/ratelimiter already exists. Investigation task: does its API expose fixed-ladder per-key state, or only token-bucket? If fixed-ladder is missing, Phase 1 contributes the ladder pattern UPSTREAM first per the Decoupled Reusable Architecture rule, then pins the new submodule hash in Lava. Lava-side internal/auth/backoff.go becomes thin glue. (If the submodule already supports the pattern, the Lava-side file is even thinner.)

7. Client-side architecture

7.1 Build-time codegen

A new Gradle task `:app:generateLavaAuthClass` runs before `compileKotlin`:

1. Read `.env` (already loaded by existing build script via `keystoreLoader` pattern).
2. Look up `LAVA_AUTH_CURRENT_CLIENT_NAME`. If not present → fail build with explicit error.
3. Find the matching entry in `LAVA_AUTH_ACTIVE_CLIENTS`. If not present → fail build.
4. Read the UUID bytes (parse the `xxxx-xxxx-...` representation into 16 raw bytes).
5. Compute the per-build encryption key. The signing-cert hash is the SHA256 of the X.509 DER bytes of the certificate embedded in the variant's keystore (`keystores/<variant>.jks`, path resolved by the existing `keystoreLoader` pattern). Build variant selects keystore: debug variant → debug keystore; release variant → release keystore. The runtime `SigningCertProvider` reads the SAME certificate from `PackageManager.getPackageInfo(... GET_SIGNING_CERTIFICATES)` — DER bytes are identical → hash is identical → derived key matches.

```
certBytes      =
readKeystore(variant).getCertificate(alias).encoded    // DER
bytes
signingCertHash = SHA256(certBytes)[:16]
pepper         = base64-
decode(LAVA_AUTH_OBFUSCATION_PEPPER)    // 32 bytes
key           = HKDF-SHA256(salt = signingCertHash, ikm =
pepper, info = "lava-auth-v1", length = 32)
```

6. AES-GCM encrypt: `(blob, nonce, tag) = AES-256-GCM(uuid_bytes, key, random_nonce_12_bytes)`
7. Generate Kotlin source `app/src/main/generated/kotlin/lava/auth/LavaAuth.kt`:

```
internal object LavaAuth {
    internal val BLOB:    ByteArray = byteArrayOf(/* 16 + tag
bytes */)
    internal val NONCE:   ByteArray = byteArrayOf(/* 12 bytes
*/)
    internal val PEPPER:  ByteArray = byteArrayOf(/* 32 bytes
*/)
}
```

```

        internal val FIELD_NAME: String = "<from
        LAVA_AUTH_FIELD_NAME>"
    }

```

8. Zero the in-memory plaintext UUID buffer used during codegen.

@Generated annotation (or moduleName-style comment) for the generator's identity. Generated file is .gitignore'd but committed-to-build-artifacts location is app/build/generated/lava-auth/.

R8 keep-rules retain LavaAuth as internal but obfuscate field names (the field names like BLOB, NONCE are decoded by the only caller — an inline crypto helper — via reflection-free direct reference).

7.2 Runtime decryption + injection

A new core/network/impl/AuthInterceptor.kt:

```

internal class AuthInterceptor @Inject constructor(
    private val signingCertProvider: SigningCertProvider,
) : Interceptor {

    override fun intercept(chain: Interceptor.Chain): Response {
        val keyBytes = ByteArray(32)
        val uuidBytes = ByteArray(16)
        try {
            // Compute the same key the build-time codegen used
            val certHash =
                signingCertProvider.sha256().copyOfRange(0, 16)
            HKDF.deriveKey(
                salt = certHash,
                ikm = LavaAuth.PEPPER,
                info = "lava-auth-v1".toByteArray(Charsets.UTF_8),
                output = keyBytes,
            )
            // AES-GCM decrypt
            AesGcm.decrypt(
                ciphertextWithTag = LavaAuth.BLOB,
                key = keyBytes,
                nonce = LavaAuth.NONCE,
                aad = null,
                output = uuidBytes,
            )
            val headerValue =
                Base64.getEncoder().encodeToString(uuidBytes)

            val req = chain.request().newBuilder()
                .header(LavaAuth.FIELD_NAME, headerValue)
                .build()
            return chain.proceed(req)
        } finally {

```

```

        keyBytes.fill(0)
        uuidBytes.fill(0)
        // headerValue is a String – JVM-immutable, can't be
        zeroed.
        // Acceptable: it leaves Lava code as soon as OkHttp
        consumes it.
        // Mitigation: do NOT log this header anywhere.
    }
}
}

```

SigningCertProvider (also new) returns the SHA-256 of the running APK's signing certificate via `PackageManager.getPackageInfo(... PackageManager.GET_SIGNING_CERTIFICATES)`. Same cert hash that the build-time codegen used → same derived key → AES-GCM decrypt succeeds. A re-signed APK has a different cert hash → derived key differs → decrypt fails → no header sent → API responds 401. This kills the re-sign-and-redistribute attack vector.

7.3 Memory-hygiene clause (core/CLAUDE.md)

Append to core/CLAUDE.md:

Auth UUID memory hygiene. The decrypted UUID inside the client MUST be held only as `ByteArray`, never as `String`. The decrypt-use-zeroize lifetime is bounded by a single `OkHttp Interceptor.intercept()` call; the `ByteArray` is `fill(0)`'d in a `finally` block before the function returns. The Base64-encoded header `VALUE` is a `String` (immutable) but never logged, never persisted, never assigned to a class field, and leaves Lava code as soon as `OkHttp` consumes it. Adding logging that includes the header value is a constitutional violation; pre-push grep enforces.

8. Transport policy

- API binds two listeners on the same logical port (8443): UDP via `quic-go` for HTTP/3, TCP via `net/http` for HTTP/2.
- Both listeners share one TLS config (`MinVersion: tls.VersionTLS13`).
- Every HTTP/2 response carries `Alt-Svc: h3=":<port>"; ma=86400` so HTTP-3-capable clients upgrade.
- Brotli response middleware applied after authMW (no point compressing 401/426 envelopes).
- Content-Encoding: `br` advertised when client `Accept-Encoding` includes `br`.

- Brotli **request** decoding off by default
(LAVA_API_BROTLI_REQUEST_DECODE_ENABLED=false); request bodies in Phase 1 are JSON {auth: ..., query: ...} shapes whose compression overhead exceeds savings. Configurable for forward compatibility.
- New Prometheus counter:
lava_api_request_protocol_total{protocol="h3"|"h2",status="2xx"|"4xx"|"5xx"}. Visible in the existing observability profile.

9. Rotation workflow

The runbook lives at docs/RELEASE-ROTATION.md (created in Phase 1):

1. uuidgen → new UUID for the upcoming release.
2. openssl rand -base64 32 → new pepper.
3. Update .env:
 - Append android-\${NEW_VERSION}-\${NEW_CODE}:\${NEW_UUID} to LAVA_AUTH_ACTIVE_CLIENTS.
 - Set LAVA_AUTH_CURRENT_CLIENT_NAME=android-\${NEW_VERSION}-\${NEW_CODE}.
 - Replace LAVA_AUTH_OBFUSCATION_PEPPER with the new value.
4. Build APK via ./build_and_release.sh (uses the new pepper + new UUID).
5. Distribute via Firebase: scripts/firebase-distribute.sh.
6. Restart API: scripts/distribute-api-remote.sh (deploys new .env to thinker.local + restarts container).
7. Wait until $\geq 95\%$ of installs (per Crashlytics version distribution) are on the new version.
8. Move the oldest entry from LAVA_AUTH_ACTIVE_CLIENTS to LAVA_AUTH_RETIRED_CLIENTS.
9. Restart API. Old APKs now receive 426 Upgrade Required → user upgrades.

If an active UUID leaks before step 7, skip to step 8 immediately for the leaked release. Affected users get 426, see upgrade dialog, install newer release.

10. α -hotfix: hide unsupported providers

Add `apiSupported`: Boolean to `core/tracker/api/.../TrackerDescriptor` (or wherever the descriptor sealed-interface lives — to be confirmed during plan phase). Initial values:

- `rutracker.org` → `true`
- `rutor.info` → `true`
- `internet-archive` → `false` (flips to `true` in Phase 2)
- Any other registered tracker → audited per descriptor

feature/onboarding's provider list filters `descriptors.where { it.apiSupported }`. Settings → Trackers also marks unsupported descriptors with a "Not yet supported via this backend" subtitle.

Existing installs with an unsupported provider already selected: `MainActivity.onCreate()` checks the persisted selected providers; if any has `apiSupported = false`, surface a one-time `AlertDialog`: *"is not currently supported via this backend. Please configure another provider."* On dismiss, navigate to Settings → Trackers. The "one-time" tracking is via a Preferences boolean flag.

This is the Phase-1 stopgap. Phase 2 makes IA `apiSupported = true` by adding the routes, and the hotfix becomes inert.

11. Constitutional addition: §6.R

Add the following to root `CLAUDE.md` after §6.Q (so the chain reads ... §6.O, §6.P, §6.Q, §6.R, §6.L):

§6.R — No-Hardcoding Mandate (added 2026-05-06, FOURTEENTH §6.L invocation)

No connection address, port, header field name, credential, key, salt, secret, schedule, algorithm parameter, or domain literal shall appear as a string/int constant in tracked source code (`.kt`, `.java`, `.go`, `.gradle`, `.kts`, `.xml`, `.yaml`, `.yml`, `.json`, `.sh`). Every such value MUST come from a config source: `.env` (gitignored), generated config class (build-time codegen reading `.env`), runtime env var, or mounted file.

The placeholder file `.env.example` (committed) carries dummy values for every variable so a developer cloning the repo knows what to set.

scripts/check-constitution.sh MUST grep tracked files for forbidden literal patterns:

- Any IPv4 address outside .env.example and incident docs
- The header name from .env.example's LAVA_AUTH_FIELD_NAME
- Any 36-char UUID outside .env.example
- Hardcoded host:port pairs in HTTP/HTTPS URLs

Pre-push rejects on match. Bluff-Audit stamp required on any commit that adds new config-driven values, demonstrating the no-hardcoding contract test fails when a literal is reintroduced.

Inheritance: applies recursively to every submodule and every new artifact. Submodule constitutions MAY add stricter rules (e.g. "this submodule MAY NOT read environment variables at all; all config arrives via constructor injection") but MUST NOT relax this clause.

Also append to AGENTS.md and every submodule's CLAUDE.md per §6.F inheritance.

12. Acceptance criteria

Phase 1 is complete when ALL of:

12.1 API-side

☐

internal/auth/middleware.go exists, exports AuthMiddleware. Active-UUID path returns 200; retired-UUID path returns 426 with min-version JSON; unknown-UUID path returns 401.

☐

internal/auth/backoff.go exists, exports Backoff + BackoffMiddleware. Per-IP fixed ladder; resets on success.

☐

cmd/lava-api-go/main.go wires both middlewares before all v1 routes.

☐

.env.example carries placeholders for every new variable.

☐

internal/config/config.go reads + validates every new variable; failure to parse is a fatal boot error (no silent defaults).

12.2 Client-side



`app/build.gradle.kts` registers `:app:generateLavaAuthClass` task; runs before `compileKotlin`; fails the build if `.env` lookup fails.



`app/src/main/generated/kotlin/lava/auth/LavaAuth.kt` is generated (gitignored; checked into build artifacts only).



`core/network/impl/AuthInterceptor.kt` exists, registered into the `OkHttp` client chain via `NetworkModule`.



`SigningCertProvider` exists, returns the running APK's signing-cert SHA256.



R8 keep-rules retain the `LavaAuth` object's bytes but not its symbol names.

12.3 Transport



HTTP/3 listener active on the configured port; `Alt-Svc` header set on every HTTP/2 response.



Brotli response middleware compresses for clients sending `Accept-Encoding: br`.



`lava_api_request_protocol_total{protocol="h3"|"h2"}` metric increments correctly under both transports.

12.4 Rotation



`docs/RELEASE-ROTATION.md` exists with the operator runbook.



`scripts/check-constitution.sh` rejects pushes that add a literal UUID outside `.env.example`.



`scripts/firebase-distribute.sh` (extended) reads `LAVA_AUTH_CURRENT_CLIENT_NAME` and refuses to distribute if the chosen UUID is not in `LAVA_AUTH_ACTIVE_CLIENTS`.

12.5 α-hotfix

- ☐ TrackerDescriptor.apiSupported: Boolean added; existing onboarding consumes it.
- ☐ First-time-only dialog surfaced for installs that already have an unsupported provider selected.
- ☐ Internet Archive descriptor: apiSupported = false in Phase 1; flips in Phase 2.

12.6 Constitutional

- ☐ \$6.R added to root CLAUDE.md, AGENTS.md, every submodules/*/CLAUDE.md.
- ☐ scripts/check-constitution.sh extended for forbidden-literal greps; pre-push hook updated to call it.
- ☐ Falsifiability rehearsal in commit body: deliberate literal reintroduction fails the checker.

12.7 Tests (§6.G real-stack + §6.A contract + §6.N rehearsals)

Test class	Path	Asserts on
AuthActiveUuidReturns200Test	lava-api-go/ tests/ integration/	real Gin engine + real Po real HTTP request → 200 correct response body
AuthRetiredUuidReturns426Test	same	retired UUID → 426 + JSC containing min_supported_version_n _code
AuthUnknownUuidReturns401Test	same	unknown UUID → 401 + c increments
AuthBackoffLadderTest	same	7 sequential 401s; assert response carries Retry-A <BACKOFF_STEPS[i]>
AuthBackoffResetsOnSuccessTest	same	

Test class	Path	Asserts on
		5 fails, 1 success, 1 fail → is at step 1 not step 6
AuthBrotliResponseTest	same	Accept-Encoding: br → r carries Content-Encoding decompressed body mat uncompressed control
AuthH3AltSvcAdvertisementTest	same	HTTP/2 response carries Svc: h3=":<port>"
AuthProtocolMetricTest	same	metric lava_api_request_protoco increments correctly per protocol used
AuthHeaderInjectionContractTest	lava-api-go/ tests/contract/	§6.A contract: LAVA_AUTH_FIELD_NAME fro .env.example matches wl internal/auth/middlewar reads (parallels healthcheck_contract_te pattern)
AuthClientFieldNameMatchesApiTest	Android side, core/network/ impl/src/ test/.../	the build-time-generated LavaAuth.FIELD_NAME mat what the API expects (cro checks .env.example)
C13_AuthHeaderInjectionTest	app/src/ androidTest/.../ challenges/	Compose UI Challenge po clause 4: real screen → re ViewModel → real OkHtt AuthInterceptor → real l go in podman → asserts c visible state ("search res loaded successfully")
C14_AuthRotationForceUpgradeDialogTest	same	Compose UI Challenge: A returns 426 → upgrade d renders + user-visible tex matches expected

For each test class added or modified, a Bluff-Audit stamp goes in the commit body per §6.N. At least one mutation per class must produce a crisp failure message.

12.8 Distribution gate (§6.I + §6.K + §6.P)

- ☐ API binary built via submodules/containers per §6.K; binary digest recorded in `.lava-ci-evidence/<tag>/api-build-digest.txt`.
- ☐ Distributed to thinker.local via `scripts/distribute-api-remote.sh`.
- ☐ APK built, signed, distributed via Firebase (Lava-Android-1.2.7-1027); §6.P CHANGELOG entry + per-version snapshot at `.lava-ci-evidence/distribute-changelog/firebase/1.2.7-1027.md`.
- ☐ §6.I emulator matrix gate green: phone + tablet on API 28/30/34/latest, all rows pass (gating: true, concurrent: 1, no AVD-shadow).
- ☐ Operator notified to install + smoke-test the four flows: login, search-against-rutacker, browse, download.

13. Risks + mitigations

Risk	Mitigation
submodules/ ratelimiter doesn't expose fixed-ladder per- key state	Plan-phase investigation; if missing, contribute upstream first per Decoupled Reusable Architecture rule (a multi-day extension before Phase 1's Lava-side work begins)
Re-signed-APK auth bypass through derived- key change	Killed by design: signing cert SHA enters HKDF salt; re-sign breaks decrypt
Per-release pepper rotation forgotten by operator	<code>scripts/firebase-distribute.sh</code> extension: refuse to distribute if <code>LAVA_AUTH_OBFUSCATION_PEPPER</code> matches the previous distribute's value (compares against <code>.lava-ci-evidence/distribute-changelog/ firebase/last-pepper.sha256</code>)
Generated <code>LavaAuth.kt</code> accidentally committed	Add to <code>.gitignore</code> ; pre-push hook scans for the file path

Risk	Mitigation
Header name leaked via logging	The constitutional core/CLAUDE.md clause + scripts/check-constitution.sh grep for the field name in non-test logging code
HTTP/3 disabled by accident on thinker.local	Boot-time validation: if LAVA_API_HTTP3_ENABLED=true but UDP listener fails, abort startup with explicit error (no silent fallback to HTTP/2-only)
Backoff state grows unbounded under brute force	sync.Map entries with blockedUntil + 24h < now are pruned by a background goroutine

14. Plan-phase investigations

Items the implementation plan (writing-plans) must resolve before phase work begins:

1. **submodules/ratelimiter capabilities audit.** Does it already expose fixed-ladder per-key backoff? Is the API stable? If yes, Phase 1's internal/auth/backoff.go is thin glue. If no, plan a small upstream contribution to submodules/ratelimiter that adds the ladder primitive (with its own §6.A contract test + §6.N falsifiability rehearsal), pin a new submodule hash in Lava, then proceed with Phase 1's Lava-side work. Per Decoupled Reusable Architecture rule: upstream first, Lava pin second.
2. **TrackerDescriptor.apiSupported location.** The descriptor sealed-interface lives somewhere under submodules/tracker_sdk or core/tracker. Plan must locate the canonical site and decide whether apiSupported is a constructor parameter or a derived property.
3. **OkHttp Interceptor registration site.** Trace the existing NetworkModule Hilt graph to confirm where AuthInterceptor is added; ensure it runs after the existing NetworkLogger (so logger never sees an unredacted header) and before TLS.
4. **Build variant + keystore wiring.** Confirm keystoreLoader pattern resolves debug/release keystores correctly; confirm Gradle task can invoke it before compileKotlin.
5. **scripts/check-constitution.sh regex set.** Plan must enumerate the exact forbidden-literal regexes for §6.R enforcement; ensure pre-push performance is acceptable on the full git ls-files scan.

6. **scripts/firebase-distribute.sh extension scope.** Confirm where the previous-pepper-SHA is read/written; ensure the same-pepper-rejection check fires reliably.

15. References

- Operator TODO: docs/todos/Lava_TODOs_001.md
- Constitution root: CLAUDE.md §6.A through §6.Q
- Existing API auth surfaces: lava-api-go/internal/auth/passthrough.go, multiprovider.go
- Existing client network: core/network/impl/src/main/kotlin/lava/network/impl/
- Memory anchor: feedback_lan_tls_no_manual_install.md (LAN TLS without manual cert install — must be preserved)
- §6.R inheritance target: every submodules/*/CLAUDE.md
- §6.L invocation count: this spec corresponds to the FOURTEENTH operator restatement of the Anti-Bluff Functional Reality Mandate (the no-hardcoding directive piggy-backs on §6.L's spirit and motivates §6.R's mechanics)